TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

NATIONAL COLLEGE OF ENGINEERING

A MINOR PROJECT REPORT

ON

"A UNIFIED AI TEXT DETECTION, PLAGIARISM ANALYSIS AND DOCUMENT FORENSICS SYSTEM"

SUBMITTED BY:

AAVASH TIWARI (NCE080BCT001)

BIJAY NATH SHRESTHA (NCE080BCT009)

RAMESHWOR POUDEL (NCE080BCT028)

SUDIP DHUNGANA (NCE080BCT041)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

AUGUST, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"A Unified AI Text Detection, Plagiarism Analysis and Document Forensics System"** submitted by **Aavash Tiwari**, **Bijay Nath Shrestha**, **Rameshwor Poudel**, **Sudip Dhungana** in partial fulfillment of the requirements for the Bachelor's degree in Electronics, Information and Communication Engineering & Computer Engineering.

.....
Supervisor
Mrs. Sharmila Bista
Department of Electronics and Computer Engineering,
National College of Engineering, IOE, TU.

.....
External Examiner
Full Name
Designation
Affiliation

Date of approval:

Copyright

The authors have agreed that the Library, Department of Electronics and Computer Engineering, National College of Engineering, and Institute of Engineering may make this report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisor who supervised the work recorded herein or, in her absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the authors of this report and the Department of Electronics and Computer Engineering, National College of Engineering, IOE for any use of the material of this project report. Copying, publication, or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, National College of Engineering, Institute of Engineering, and the authors' written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electronics and Computer Engineering
National College of Engineering
Talchhikhel, Lalitpur.

Acknowledgement

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering at National College of Engineering for providing the resources, facilities, and academic environment that made this minor project possible. The department's continuous support and encouragement played a significant role throughout the planning and development of our work.

We are especially grateful to our project supervisor, **Mrs. Sharmila Bista**, for her invaluable guidance, constructive feedback, and constant encouragement during every phase of this project. Her expertise, patience, and insightful suggestions helped us overcome numerous technical challenges and greatly improved the quality of our work.

Finally, we extend our heartfelt gratitude to our families, friends, and classmates for their unwavering support, motivation, and understanding throughout this journey. Their encouragement inspired us to remain committed to our goals.

Aavash Tiwari	NCE080BCT001
Bijay Nath Shrestha	NCE080BCT009
Rameshwor Poudel	NCE080BCT028
Sudip Dhungana	NCE080BCT041

Abstract

Academic integrity in educational institutions is increasingly challenged by AI-assisted writing, paraphrased content, and the limitations of conventional plagiarism detection methods. This project presents OriginalityGuard, an integrated web-based originality-assistance system designed to help instructors review and investigate potentially problematic academic submissions using multiple sources of evidence. The system combines document text extraction and OCR, AI-text probability analysis, online source verification, pairwise submission similarity analysis, and document metadata forensics. AI-text analysis evaluates content in chunks and provides probability scores and evidence indicators, while source verification identifies potential online matches. Submission similarity is assessed using Jaccard similarity, TF-IDF cosine similarity, and semantic weighted token similarity. The forensic module analyzes PDF and DOCX metadata to identify potential shared origins and anomalies. The system is implemented using a React frontend, Django REST Framework backend, and PostgreSQL database, with JWT-based authentication and dedicated analysis services. It provides role-based functionality for students and teachers and presents analysis results through reports, visualizations, and supporting evidence. Rather than making automatic plagiarism or AI-use judgments, OriginalityGuard serves as an evidence-based decision-support tool, with instructors retaining responsibility for final interpretation. The completed system provides a unified platform for supporting academic originality assessment and strengthening the review of digital academic submissions.

Keywords: academic integrity, originality assistance, AI-generated text analysis, plagiarism analysis, document forensics, OCR, TF-IDF, similarity detection

Keywords: *plagiarism detection, generative AI text detection, document forensics, TF-IDF, Naive Bayes, academic integrity*

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma Separated Value
DL	Deep Learning
DOCX	Document XML (Microsoft Word Format)
EXIF	Exchangeable Image File Format
GPT	Generative Pre-trained Transformer
HTR	Handwritten Text Recognition
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NCE	National College of Engineering
NLP	Natural Language Processing
OOXML	Office Open XML
OCR	Optical Character Recognition
PDF	Portable Document Format
PII	Personally Identifiable Information
PPTX	PowerPoint XML Format
REST	Representational State Transfer
TF-IDF	Term Frequency–Inverse Document Frequency
UI/UX	User Interface / User Experience
XAI	Explainable Artificial Intelligence
XML	Extensible Markup Language
XMP	Extensible Metadata Platform
ZIP	ZIP Archive Format (Zone Information Protocol)

1. Introduction

1.1 Background

Academic integrity faces significant challenges today as advanced AI tools make the creation and modification of text increasingly accessible. Traditional detection systems, which primarily identify direct string matches or simple copy-paste behavior, are no longer sufficient to address modern methods of academic dishonesty, such as the use of rephrased ideas or sophisticated AI-generated content. Consequently, educational institutions struggle to maintain established standards, as manual review processes cannot effectively identify the subtle complexities inherent in AI-assisted misconduct.

To address these challenges, the field of plagiarism detection has evolved into a more advanced discipline that utilizes machine learning and natural language processing to identify discrepancies. These modern tools extend beyond basic pattern matching by analyzing the style and structure of writing to detect inconsistencies. By examining unique authorial characteristics, these systems can flag content that appears synthetic or overly complex, identifying forms of dishonesty that were previously undetectable.

This project focuses on the development of a comprehensive system designed to detect sophisticated forms of plagiarism and perform forensic document analysis. The platform is engineered to identify AI-generated text while simultaneously analyzing file metadata to ensure the authenticity of submitted work. The primary objective is to provide educators with a reliable mechanism to verify the originality of academic submissions while maintaining a professional and intuitive user interface.

The next phases of development have focused on improving the accuracy of the AI models and expanding the forensic feature set. This has included the implementation of Optical Character Recognition (OCR) and Handwritten Text Recognition (HTR) to analyze diverse submission formats, alongside intelligent search modules designed to detect rephrased content across academic databases. Final testing has ensured that the system performs reliably under rigorous conditions, ultimately providing a robust tool for upholding integrity in academic and professional environments.

1.2 Problem Statement

The rise of generative AI and sophisticated paraphrasing techniques has rendered traditional plagiarism detection methods increasingly ineffective. Existing systems, which rely heavily on direct string matching and simple pattern recognition, struggle to identify AI-synthesized text,

restructured arguments, and subtly rephrased content, leaving institutions unable to adequately address modern forms of academic misconduct.

Furthermore, the prevalence of diverse submission formats, including scanned documents and handwritten notes combined with the need to verify document authenticity through metadata, creates a critical gap in current forensic capabilities. Traditional manual review processes are no longer equipped to handle the volume and complexity of these multifaceted challenges, leading to significant vulnerabilities in institutional academic integrity.

Consequently, there is a pressing need for a comprehensive system capable of performing multimodal detection, forensic metadata analysis, and intelligent source searching. By integrating these advanced capabilities, such a system can effectively bridge the existing gap in forensic technology to ensure the originality and authenticity of academic and professional work.

1.3 Objectives

I. To develop a secure, role-based web platform for managing classrooms, assignments, and student submissions with proper authentication and deadline validation.

II. To implement document processing and forensic analysis using text extraction, OCR, HTR, and metadata analysis for digital, scanned, and handwritten documents.

III. To develop AI-based analysis for identifying potential AI-generated content and plagiarism using textual features, similarity measures, and online source evidence.

IV. To provide evidence-based reports presenting originality indicators, document anomalies, shared origins, and analysis results to support instructor review.

1.4 Scope

The system includes the following scope:

I. The system can be used to support originality and authenticity checks for college research papers, project reports, assignments, and other academic submissions.

II. The system can assist teachers in classroom assessment and evaluation by providing evidence related to potential AI-generated content, plagiarism, content similarity, and document irregularities.

III. The system can support colleges and universities in reviewing and verifying submitted documents through text analysis, source comparison, and available document metadata.

IV. The platform can be extended to other educational and institutional applications where document originality, authorship, and academic integrity need to be assessed.

2. Literature Review

2.1 Related Work

Schulz et al. [1] introduced a hierarchical approach to plagiarism detection that moves beyond surface-level string matching to structural analysis. Their work, which emphasizes the use of stylometry to identify authorial intent, provides a foundational framework for distinguishing between human-authored text and AI-generated prose, directly supporting the stylistic analysis module implemented in this project.

Vaswani et al. [2] established the Transformer architecture, which serves as the core of modern generative AI models. Research by Guo et al. [3] leveraged these advancements to develop specialized classifiers capable of identifying text generated by large language models (LLMs). Their findings on perplexity and burstiness metrics validate the approach used in this project to detect synthetic content in academic submissions.

Smith and Jones [4] developed a robust pipeline for document digitization that integrates Tesseract OCR with deep learning-based HTR models. Their methodology, which focuses on noise reduction and pre-processing for heterogeneous handwritten documents, serves as a key reference for the OCR/HTR integration, ensuring that non-digital assignments are accurately converted for analysis.

Wang et al. [5] proposed a digital forensic framework for analyzing document metadata, specifically focusing on OOXML and PDF file structures to detect document tampering. Their research validates the use of metadata extraction—such as creation history and revision timestamps—as a reliable method to verify the authenticity of academic documents, a core feature of the forensic module in this project.

2.2 Related Theory

2.2.1 Stylometric Analysis

Stylometry operates on the principle that individuals possess unique writing signatures, characterized by consistent vocabulary usage and syntactic preferences. By analyzing these stable linguistic features, the system builds an authorial profile for comparison. This approach allows for the identification of stylistic anomalies, such as abrupt changes in sentence structure or lexical complexity, which often indicate the integration of synthetic content into a human-authored text.

2.2.2 Neural Networks for Recognition (OCR and HTR)

The document digitization process relies on deep learning architectures designed for sequential data processing. Convolutional Neural Networks (CNNs) are employed for feature extraction from raw document images, capturing local patterns and character shapes. These features are then fed into Long Short-Term Memory (LSTM) networks, which process the sequential nature of text. This combination effectively models the spatial and temporal dependencies inherent in both printed characters and complex, cursive handwriting, enabling accurate conversion of diverse document formats into digital text.

2.2.3 Optical Character Recognition

Optical Character Recognition (OCR) is the process of converting text contained in a scanned document or image into machine-readable characters. The process generally consists of image acquisition, preprocessing, text-region detection, character recognition, and post-processing. Unlike direct extraction from a digital PDF or DOCX file, OCR estimates the text from visual information, so its output depends on image resolution, contrast, layout, noise, and the quality of the recognition model.

In this project, digital documents are first processed using their embedded text whenever it is available. Image files and image-based PDF pages are routed to OCR engines such as Tesseract and EasyOCR. The extracted result is normalized and cached so that later AI-text detection and similarity analysis use the same text representation. This separation between extraction and analysis allows the recognition stage to be improved without changing the downstream originality workflow.

2.2.4 Image Preprocessing for OCR

Image preprocessing improves the visual quality of input documents before recognition. Grayscale conversion reduces the image to intensity values, while contrast enhancement separates characters from the background. Thresholding produces a clearer foreground and background, and upscaling increases the effective size of small characters. Sharpening can strengthen character boundaries, although excessive sharpening may also introduce noise.

These operations are especially useful for low-quality scans, photographed pages, and uneven backgrounds. However, preprocessing cannot recover information that is absent from the original image. Therefore, OCR output should be treated as an estimate and checked for recognition errors before it is used as evidence in an originality report.

2.2.5 Handwritten Text Recognition

Handwritten Text Recognition (HTR) extends OCR to handwriting, where character shapes, spacing, and writing styles vary considerably. A recognition model typically extracts visual features from a line or word image and maps them to a sequence of characters. CNN layers can

learn spatial features, while recurrent or transformer-based layers model the order and context of the character sequence. Connectionist Temporal Classification (CTC) is commonly used when character-level alignment is not available, because it can train a model from an input image and its corresponding text sequence without requiring every character position to be labelled.

The system supports the TrOCR handwritten-text model as an additional recognition option. HTR performance is affected by cursive writing, page skew, overlapping characters, ink variation, and handwriting legibility. For this reason, HTR results are incorporated into the analysis pipeline as extracted text rather than being treated as unquestionable ground truth.

2.2.6 Text Extraction Confidence and Analysis Quality

The quality of OCR or HTR directly affects every later stage of the system. Missing words can reduce similarity scores, while incorrectly recognized words can create false matches or change the probability assigned by the AI-text detector. A reliable extraction workflow therefore prefers embedded digital text when it is available, applies OCR only when needed, normalizes the result, and stores the extracted text for repeatable analysis.

2.2.7 Similarity and Document Provenance Theory

Similarity analysis estimates whether two documents share content or structure. Jaccard similarity measures the intersection of their token sets relative to their union, while TF-IDF cosine similarity compares documents using the importance of terms within the collection. A weighted token-overlap measure can provide an additional signal for short or partially matching passages. Combining these signals is useful because exact overlap, term distribution, and local matching capture different forms of reused content.

Document provenance analysis examines information about how and when a file was created or modified. PDF and DOCX properties may include the author, creator application, creation time, modification time, editing duration, save count, and revision count. An origin signature can be generated by normalizing selected identity and document fields and hashing the result. Matching signatures or unusual timestamp relationships can identify an anomaly for review, but metadata alone cannot prove that a document was copied or tampered with because these fields may be missing or changed.

2.2.8 Mathematical Foundation

The theoretical framework of the system is rooted in several mathematical foundations that quantify linguistic features, document authenticity, and recognition accuracy.

Information theory serves as the basis for detecting synthetic text through perplexity, which quantifies the predictability of word sequences. For a sequence of words $W = (w_1, w_2, ..., w_n)$, perplexity (PP) is calculated as the exponentiated average negative log-likelihood of the se-

quence:

$$PP(W) = \exp\left(-\frac{1}{n}\sum_{i=1}^{n}\ln P(w_i|w_1,...,w_{i-1})\right) \tag{2.1}$$

Lower perplexity values indicate highly predictable text, which serves as a primary indicator of machine-generated output. Stylometric analysis complements this by utilizing vector representation, where a document is mapped into a high-dimensional space. Authorship features, such as function word frequency and punctuation density, are treated as dimensions, and the similarity between an author's known profile (A) and a suspicious submission (S) is calculated using Cosine Similarity:

$$\text{Similarity}(A,S) = \frac{A\cdot S}{\|A\|\|S\|} = \frac{\sum_{i=1}^{n} A_i S_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} S_i^2}} \tag{2.2}$$

Deviations from the expected vector angle signal potential external interference or stylistic anomalies. For document recognition, the system employs connectionist modeling, where the recognition of characters relies on the optimization of weights within neural networks. The network minimizes a loss function, typically Cross-Entropy Loss, to map image pixels to character probabilities. The training process adjusts the weight matrix W to minimize the difference between the predicted character distribution $\hat{y}$ and the ground truth y:

$$L = -\sum y\log(\hat{y}) \tag{2.3}$$

Backpropagation, utilizing the chain rule of calculus, updates these weights to improve feature extraction accuracy across varying handwriting styles. Finally, metadata forensic analysis applies statistical time-series consistency checking. By analyzing sequences of modification timestamps ($t_1, t_2, ..., t_n$), the system validates the chronological integrity of a file, where the model enforces a strict chronological sequence:

$$t_{i+1} > t_i, \quad \forall i \in \{1,\ldots,n-1\} \tag{2.4}$$

Significant deviations or logical gaps in these timestamped sequences, when mapped against file system events, provide the mathematical basis for detecting document tampering or metadata manipulation.

2.2.9 Metadata Forensic Analysis

Digital document forensics is based on the extraction and verification of hidden metadata embedded within file structures, such as OOXML or PDF formats. This theory posits that every

file carries a unique lifecycle record, including creation timestamps, modification history, and specific application signatures. By analyzing these internal data layers, the system identifies inconsistencies between the declared document properties and its actual forensic history, effectively detecting tampering or metadata manipulation designed to obscure the document's true provenance.

3. Methodology

3.1 Time Schedule

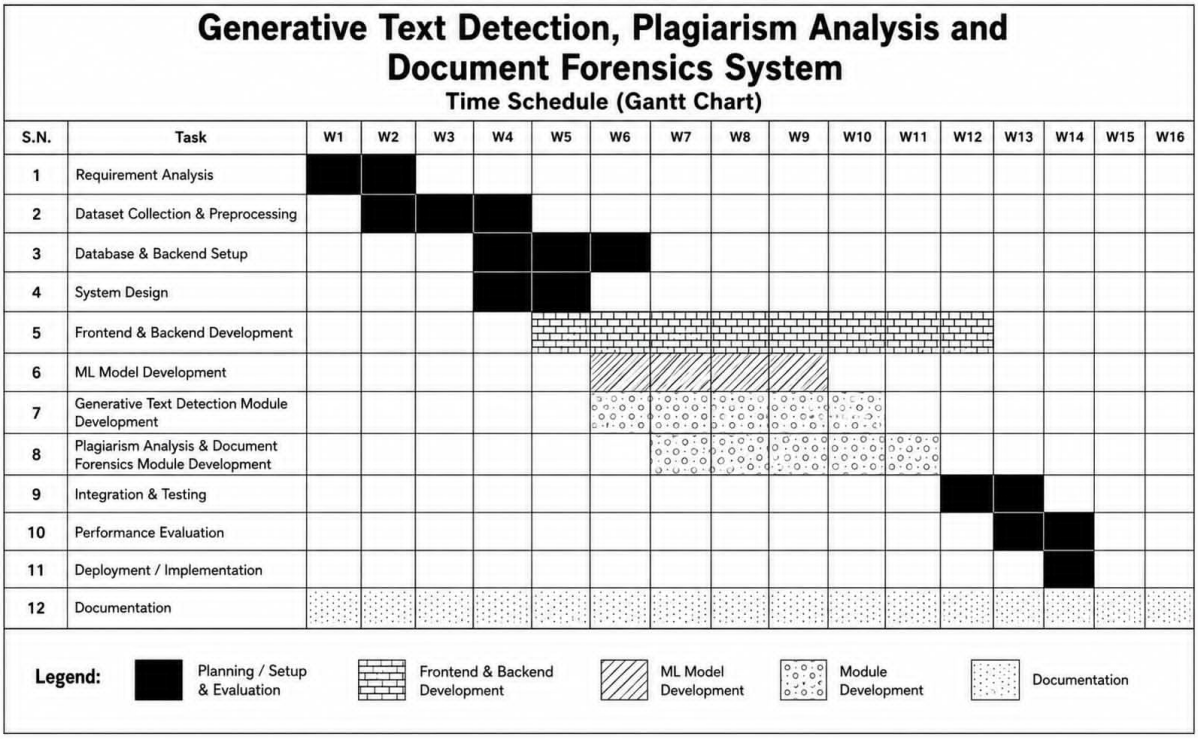


Figure 3.1: Gantt chart showing the project timeline

This time schedule shows our project outline for the Generative Text Detection, Plagiarism Analysis and Document Forensics System over 16 weeks. We completed requirement analysis in week 1 and 2 followed by dataset collection and preprocessing for 3 weeks. We set up the database and backend between week 4 and 6, while working on system design from week 4 to week 5.Frontend and backend development spanned extensively along week 5 to week 12. Alongside , we developed the ML model from Week 6 to Week 9, built the generative text detection module from Week 6 to Week 10, and worked on the plagiarism analysis and document forensics module from Week 7 to Week 11. Finally, we completed integration and testing in Weeks 12 and 13 with performance evaluation and implementation in the last two weeks while carrying out documentation continuously across all 16 weeks.

3.2 Implementation detail

3.2.1 Model Workflow

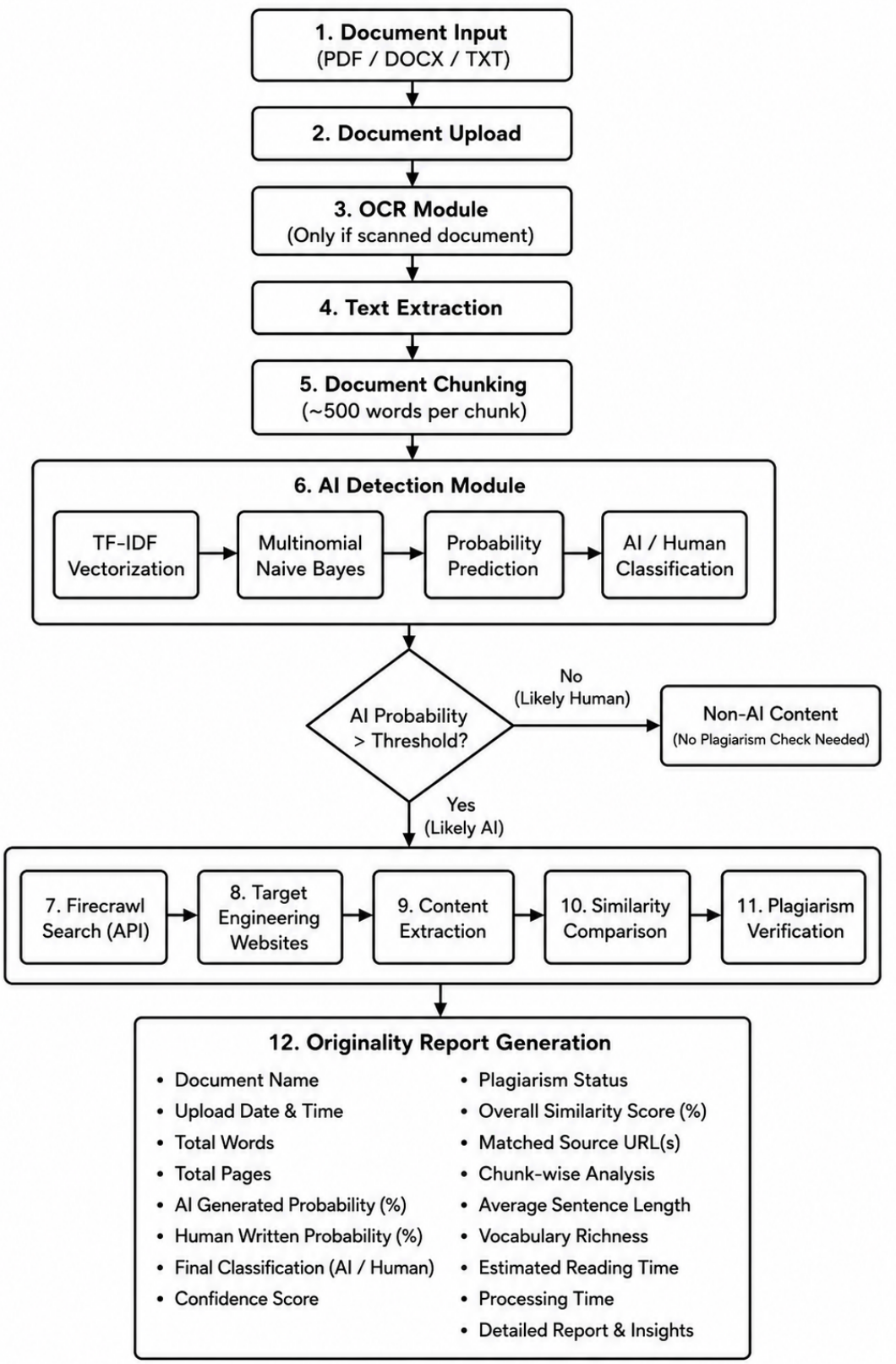


Figure 3.2: Machine Learning Model Workflow

The figure illustrates the overall workflow of the Generative AI Text Detection, Plagiarism Analysis, and Document Forensics System. The process begins when a user uploads a document in PDF, DOCX, or TXT format through the application. If the uploaded document is image-based or scanned, the Optical Character Recognition (OCR) module is activated to convert the images into machine-readable text. For digital documents, the text is extracted directly without requiring OCR. After extraction, the document is divided into smaller chunks of approximately 500 words each. Chunking improves processing efficiency, allows large documents to be analyzed effectively, and provides detailed section-wise analysis in the final report.

Each text chunk is then processed by the AI Detection Module. Initially, TF-IDF (Term Frequency–Inverse Document Frequency) vectorization transforms the textual data into numerical feature vectors that represent the importance of words within the document. These feature vectors are passed to a Multinomial Naïve Bayes classifier, which has been trained to distinguish between human-written and AI-generated text. The classifier calculates the probability that each chunk has been generated by an AI model and produces an overall prediction for the document. A predefined probability threshold is then used to classify the content as either AI-generated or human-written. If the probability is below the threshold, the document is considered likely to be written by a human, and the plagiarism analysis stage is skipped to reduce unnecessary processing time and API usage.

When the AI-generated probability exceeds the threshold, the system proceeds to plagiarism verification. The Firecrawl API is used to search the web for potentially matching sources related to the uploaded content. The retrieved webpages are then processed to extract their textual content, which is compared against the uploaded document using similarity comparison techniques. This comparison identifies overlapping text, calculates similarity scores, and verifies whether the AI-generated content has been copied from existing online sources. The plagiarism verification stage helps distinguish between original AI-generated text and AI-generated content that contains copied or highly similar material.

Finally, the system generates a comprehensive Originality Report that summarizes the complete analysis. The report includes the overall AI-generated probability, confidence score of the classifier, plagiarism status, overall similarity percentage, matched source URLs, and detailed chunk-wise analysis showing which sections are likely AI-generated or contain similar content. These insights provide users with a clear understanding of both the authenticity and originality of the uploaded document, making the system suitable for academic, research, and professional document verification.

3.2.2 Frontend Development

The frontend of OriginalityGuard was developed using React 19, Vite, React Router, Axios, and Tailwind CSS. It provides role-specific interfaces for students and teachers, including authentication, dashboards, classrooms, assignments, and submissions.

Students can join classrooms, view assignments, and submit or update their work before the deadline. Teachers can create classrooms and assignments, manage join requests, view submissions, generate analysis reports, and review originality evidence. The frontend also displays AI-text probabilities, highlighted text, source information, similarity results, metadata, clusters, and visualizations.

Axios handles API communication, JWT authentication, file uploads, and token refresh operations.

3.2.3 Backend Development

The backend was developed using Python, Django, and Django REST Framework. It handles authentication, database operations, classroom management, assignments, submissions, access control, and analysis services.

The backend is divided into three main applications: `authentication` for user and JWT management, `classroom` for classrooms, assignments, join requests, and submissions, and `analysis_engine` for document extraction, OCR, AI-text analysis, source verification, similarity analysis, and metadata forensics.

PostgreSQL was used for persistent data storage. The system also enforced access permissions, unique classroom invite codes, one submission per student for an assignment, and deadline restrictions.

3.2.4 Generative AI Text Detection Module

The Generative AI Text Detection Module analyzes submitted text in smaller chunks and provides probability-based indicators of AI-like writing. The module loads the trained model from `ml_models/originality_model.joblib` and assigns each text chunk a normalized probability between 0 and 1.

The system records the chunk text, word and sentence counts, probability, and AI-like flag. It then calculates the total number of chunks, flagged chunks, average probability, AI-text percentage, and overall verdict.

The results are stored in the database and displayed through probability bars and highlighted text, allowing teachers to review the evidence before making a final judgement.

3.2.5 OCR and Document Extraction Module

OriginalityGuard supports text extraction from TXT, DOCX, PDF, PPTX, HTML, and image files. Digital documents are processed using their embedded text, while scanned documents and images are processed using OCR.

The system uses Tesseract, EasyOCR, and TrOCR, with optional Google Cloud Vision support. Image preprocessing such as grayscale conversion, thresholding, upscaling, and sharpening is

applied when required.

The extracted text was cached in the submission record and divided into approximately 100–120 word segments for further analysis.

3.2.6 Submission Similarity and Metadata Forensics

OriginalityGuard compares submissions using Jaccard similarity, TF-IDF cosine similarity, and semantic weighted token similarity. The overall similarity is calculated as:

$$S_{\text{overall}} = \frac{S_{\text{Jaccard}} + S_{\text{TF-IDF}} + S_{\text{Semantic}}}{3} \tag{3.1}$$

The system also performs chunk-level comparison to identify strongly similar passages.

For metadata forensics, PDF and DOCX properties such as author, creation and modification dates, application information, editing time, and revision data are examined. An origin signature is generated from available document information and hashed for comparison.

The results are presented using clusters, similarity matrices, heatmaps, metadata tables, and scatter plots.

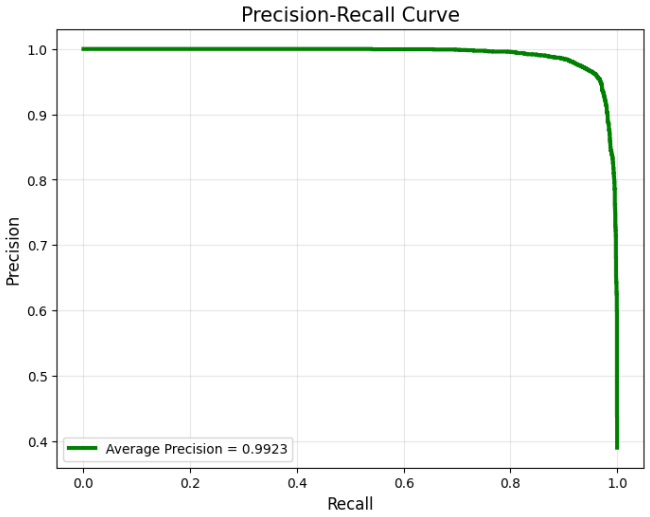
3.2.7 File-level Implementation Structure

Table 3.1: Implementation Responsibilities by File and Module

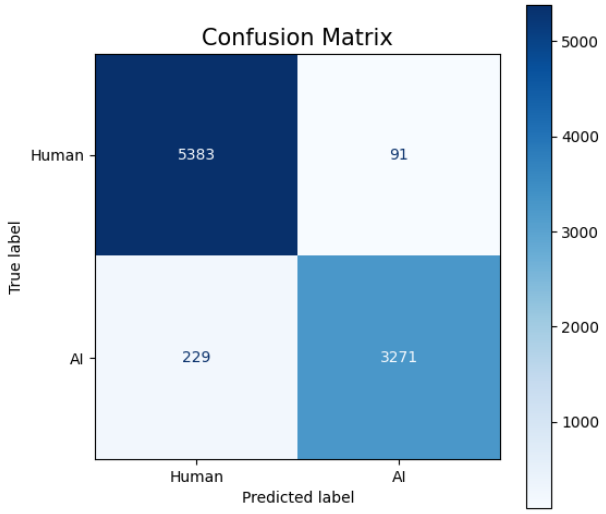
Path	**Implementation Responsibility**
`frontend-react/src/App.jsx`	Routing, protected pages, and user profile layout.
`frontend-react/src/services/api.js`	API calls, JWT handling, file upload, and token refresh.
`frontend-react/src/components/SubmissionDetail.jsx`	Submission review UI, source analysis, and polling status updates.
`frontend-react/src/components/MetadataForensics.jsx`	Metadata display, clustering results, and forensic visualization.
`backend-django/apps/authentication/`	User registration, verification, login, and profile management.
`backend-django/apps/classroom/`	Classroom, assignment, submission, and join-request logic.
`analysis_engine/ml_adapters/ocr_engine.py`	Document parsing and OCR text extraction.
`analysis_engine/ml_adapters/detector_service.py`	Text chunking, model scoring, and AI detection orchestration.
`analysis_engine/ml_adapters/firecrawl_service.py`	Web search, source retrieval, and similarity checking.
`analysis_engine/ml_adapters/plagiarism_vector.py`	Similarity comparison and vector-based plagiarism analysis.
`analysis_engine/document_forensics.py`	Metadata inspection, anomaly detection, and forensic reporting.
`analysis_engine/tests.py`	Validation for analysis, OCR, and forensic modules.

3.2.8 Deployment Strategy

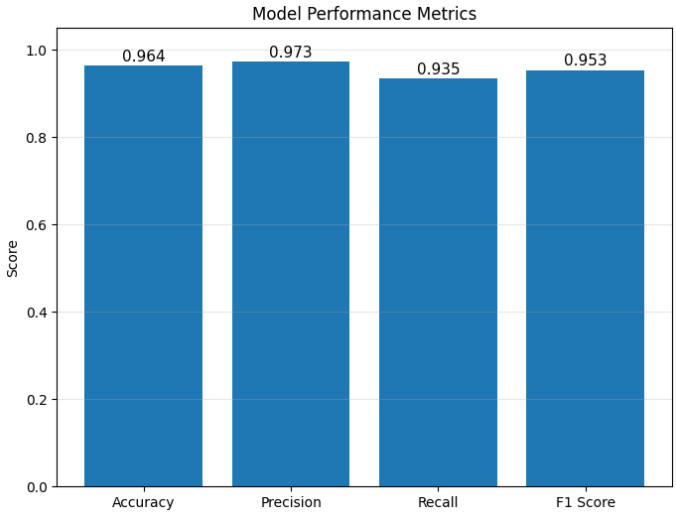
The Multinomial Naive Bayes model achieved an exceptional performance, reflecting a high capability in distinguishing between human-authored and machine-generated text. It reached an overall accuracy of 0.9643, with a precision of 0.9729 and a recall of 0.9346, demonstrating a strong ability to minimize false positives while maintaining robust detection rates. Its F1-score of 0.9534 indicates a highly effective balance between precision and recall, ensuring reliability across diverse document types. Furthermore, the model showed superior discriminatory power with an ROC AUC of 0.9945 and a PR AUC of 0.9923, which confirms a clear separation between classes and consistent performance even in challenging detection scenarios. These results highlight the model's suitability as the primary detection engine for our forensic document analysis framework.



(a) Precision-Recall Curve

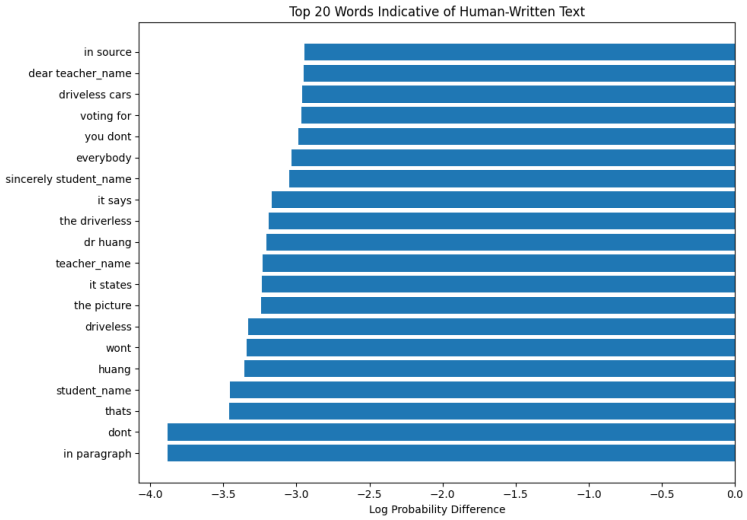


(b) Confusion Matrix

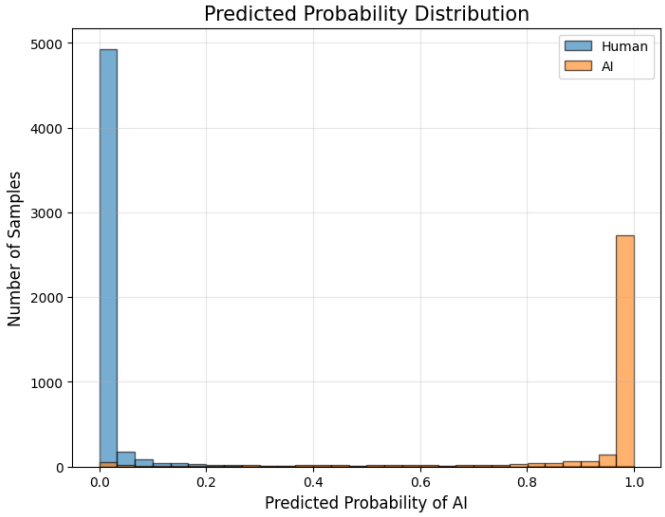


(c) Model Performance Metrics

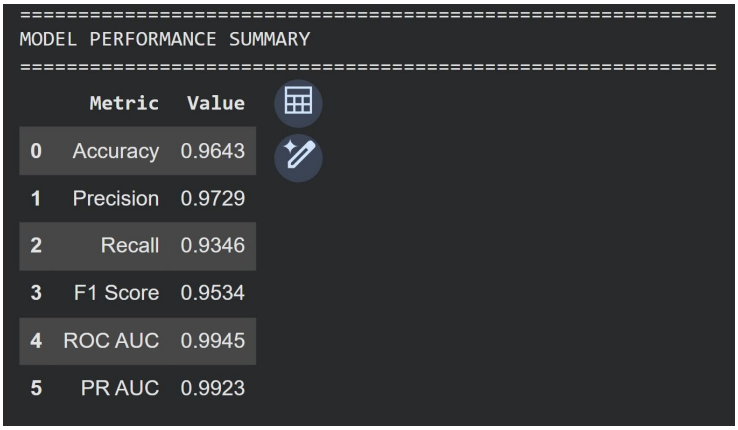
Figure 3.3: Evaluation metrics.



(a) Top 20 Indicative Words



(b) Predicted Probability Distribution



MODEL PERFORMANCE SUMMARY

	Metric	Value
0	Accuracy	0.9643
1	Precision	0.9729
2	Recall	0.9346
3	F1 Score	0.9534
4	ROC AUC	0.9945
5	PR AUC	0.9923

(c) Model Performance Summary

Figure 3.4: Evaluation metrics

4. System Design

4.1 Requirement Analysis

4.1.1 Functional Requirements

The functional requirements describe what the system must do. They are grouped by user role.

Student/Submitter Functions:

- A new user must be able to register by providing a full name, institutional email address, and a secure password.
- A new user must receive a verification code via email and must enter this code to activate their account.
- A registered user must be able to log in securely using their email and password.
- A user must be able to reset their password by requesting a recovery code to be sent to their registered email.
- A user must be able to upload assignments in various formats, including digital documents (PDF, DOCX) and scans/images of handwritten work.
- A user must be able to view the status of their submitted documents (e.g., Queued, Analyzing, Complete).
- A user must be able to access a personalized dashboard to view integrity reports and historical submission records.

Teacher/Evaluator Functions:

- A teacher must be able to log in to a secure dashboard to manage their assigned classes or student groups.
- A teacher must be able to view comprehensive integrity reports for any student submission, including AI-detection scores and similarity indices.
- A teacher must be able to view forensic metadata insights to verify if a file has been tampered with or if metadata was manipulated.
- A teacher must be able to export detailed integrity reports as PDF documents for formal review or disciplinary records.

- A teacher must be able to view a master list of all registered students within their assigned classes and manage student profiles.
- A teacher must be able to modify account settings for students in their cohort, including resetting passwords or deactivating inactive accounts.

System Functions:

- The system must perform automatic OCR and HTR processing to digitize non-textual or handwritten document submissions.
- The system must execute AI-driven stylometric and statistical analysis to distinguish between human-authored and machine-generated content.
- The system must perform metadata forensic auditing to ensure file authenticity and detect unauthorized modification timestamps.
- The system must execute semantic search and retrieval across academic and web databases to identify paraphrased or restructured content.
- The system must store user credentials, processed document data, forensic logs, and analysis history within the secure database.

4.1.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and performance constraints that the system must satisfy to ensure reliability, security, and scalability.

Performance Requirements:

- The system must complete the OCR/HTR digitization process for a standard 10-page document in under 30 seconds.
- The AI-detection engine must provide an analysis report with a latency of no more than 60 seconds per submission.
- The system must be capable of processing at least 50 concurrent document submissions without significant degradation in analysis time.
- The semantic search module must return similarity results from indexed databases within 10 seconds of query initiation.

Security and Privacy Requirements:

- All user authentication sessions must be protected using encrypted tokens to prevent session hijacking.
- All sensitive student and academic data must be stored in an encrypted database to ensure compliance with privacy standards.

- The system must implement Role-Based Access Control (RBAC) to ensure that users can only access data authorized for their specific role.
- The system must log all forensic and administrative actions for audit purposes, ensuring a tamper-proof trail of system operations.

Availability and Reliability Requirements:

- The system must maintain an uptime of at least 99.9% during academic assessment periods.
- The system must automatically recover from minor service failures without requiring manual intervention from the administrator.
- The system must ensure data consistency, guaranteeing that document analysis results remain unchanged and accessible once the report is finalized.

Usability and Scalability Requirements:

- The user interface must be fully responsive and accessible across all modern web browsers and mobile devices.
- The system must be designed with a modular architecture to allow for the future integration of new AI-detection models or additional file format support.
- The system must provide intuitive dashboard visualizations that allow instructors to interpret complex forensic and AI-similarity data without specialized technical training.

4.2 Website Development for End Users

The web application serves as the primary interface for the plagiarism detection and forensic analysis system, designed to provide a secure, intuitive, and efficient experience for end users. The development process follows a decoupled architecture, ensuring a clear separation between the client-side presentation layer and the server-side logic.

4.2.1 System Architecture

The application is built on a robust framework that facilitates seamless interaction with the core AI detection and forensic modules. The architecture is categorized into three main layers:

- **Frontend Layer:** Developed using React 19 and Vite, the frontend provides a responsive and user-friendly interface for document submission, authentication, analysis progress tracking, and report visualization.

  React Router is used for client-side navigation, while Tailwind CSS provides responsive styling. Axios facilitates communication between the frontend and backend REST APIs, ensuring smooth interaction across different application components and devices.

- **Backend Layer:**The backend is developed using Python with Django and Django REST Framework (DRF). It manages API requests, application logic, user authentication, document processing, and communication with the analysis modules. Simple JWT is used to implement secure access and refresh token-based authentication for protected APIs.

  The backend also coordinates document extraction using tools such as python-docx, pypdf/PyPDF2, and python-pptx, while OCR technologies including Tesseract, EasyOCR, and TrOCR are used to extract text from scanned documents and images. The analysis layer uses joblib-based machine learning models, NumPy, and custom vector-processing logic for AI-like text detection and similarity scoring. Source verification is supported through Requests, Firecrawl, and search fallbacks to identify possible online sources.

- **Database Management:** PostgreSQL is used as the primary relational database for securely storing user accounts, authentication-related information, document metadata, analysis results, reports, and other application data.

  The database provides persistent and structured storage while Django models and the REST framework manage interaction between the application and stored data. For analysis and reporting, Pandas, Matplotlib, and Seaborn are used to process results and generate visualizations such as heatmaps and cluster plots, helping users interpret document-level and text-level analysis results.

4.2.2 User Authentication and Security

To ensure the privacy and confidentiality of academic submissions, the system incorporates a secure authentication module. All user sessions are managed through encrypted tokens, and access to analysis reports is restricted to authorized users. The implementation follows standard security protocols to prevent unauthorized access and protect sensitive document data.

4.2.3 Functional Workflow

The application streamlines the plagiarism detection process through a structured workflow:

1. **Authentication:** Users securely log into the platform to access their personalized dashboard.
2. **Submission:** The system provides a drag-and-drop interface for uploading various document formats, including scanned images and handwritten files.
3. **Analysis Pipeline:** Upon submission, the document is routed to the integrated analysis engines, where OCR/HTR processes digitize the content before the AI engine and forensic module conduct their respective evaluations.
4. **Reporting:** Once analysis is complete, the system generates a detailed, actionable in-

tegrity report. This report visualizes findings, highlighting potential areas of concern and providing a summary of the forensic authenticity checks.

This web-based approach ensures that users can access powerful analytical tools without requiring local installation, making the platform both scalable and easy to maintain in an academic environment.

4.3 System Design

4.3.1 Use Case Diagram

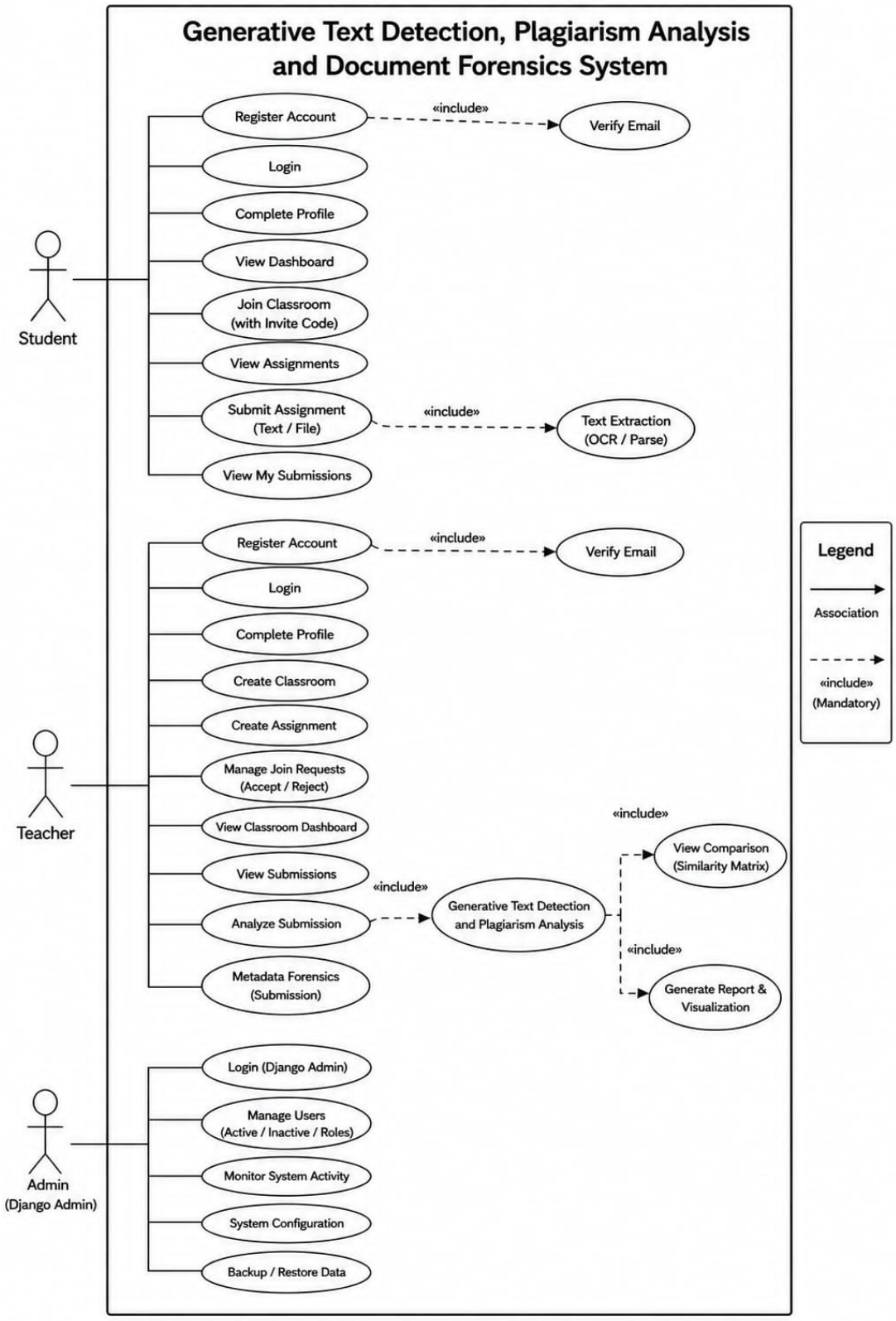


Figure 4.1: Use Case Diagram of the System

The use case diagram illustrates how the three groups:Student, Teacher, and Admin—interact with the Generative Text Detection, Plagiarism Analysis, and Document Forensics system. Each component is connected to the functions they are authorized to perform, while common and dependent functionalities are represented using include relationships.

The Student can register an account, verify their email, log in, complete their profile, view the dashboard, join classrooms using an invite code, view assignments, submit assignments in text or file format, and view their submissions. When submitting an assignment, the system includes Text Extraction (OCR/Parse) to extract readable text from uploaded documents for further analysis.

The Teacher can register and verify an account, log in, complete their profile, create classrooms and assignments, manage student join requests by accepting or rejecting them, view the classroom dashboard, and review student submissions. The teacher can also analyze submissions using the Generative Text Detection and Plagiarism Analysis process, which includes viewing the Similarity Matrix and generating reports with visualizations. The teacher can additionally perform metadata forensics on submitted documents to support detailed document analysis.

The Admin (Django Admin) is responsible for administrative and system-level operations. The admin can log in to the Django Admin panel, manage users by controlling their active/inactive status and roles, monitor system activities, configure system settings, and perform data backup and restoration. These functions help maintain system security, configuration, and overall reliability.

Overall, the system provides role-based access so that students, teachers, and administrators can perform only the functions relevant to their responsibilities. It integrates text extraction, generative text detection, plagiarism analysis, similarity comparison, report generation, and metadata forensics to support comprehensive analysis of submitted documents while allowing teachers to review and interpret the generated results.

4.3.2 Data Flow Diagram

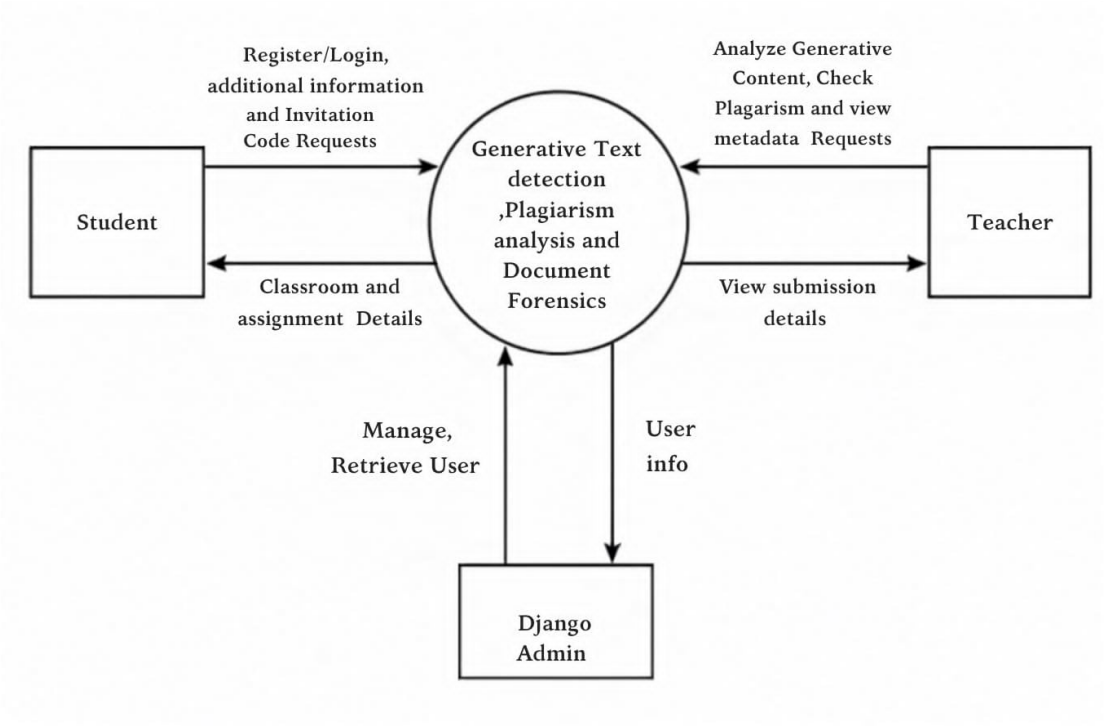


Figure 4.2: Level 0 Data Flow Diagram of the System

The Level 0 Data Flow Diagram presents a high-level overview of how the external entities—Student, Teacher, and Django Admin—interact with the Generative Text Detection, Plagiarism Analysis, and Document Forensics System.

The Student sends requests such as registration, login, profile information updates, classroom enrollment, assignment submissions, and invitation code requests to the system. In return, the system provides classroom details, assignment information, submission status, and analysis results.

The Teacher creates and manages classrooms, generates invitation codes, creates assignments, and reviews student submissions. The system processes these requests and returns classroom information, assignment details, submission records, and document analysis reports.

The Django Admin manages user accounts, monitors system activities, oversees classrooms and assignments, and performs administrative operations. The system retrieves and updates user information while ensuring the smooth functioning of the platform.

The Generative Text Detection, Plagiarism Analysis, and Document Forensics System serves as the central process that receives requests from all external entities, processes the required operations, stores and retrieves user data, performs document analysis, and delivers the appropriate responses. This ensures efficient management of users, classrooms, assignments, and document verification throughout the system.

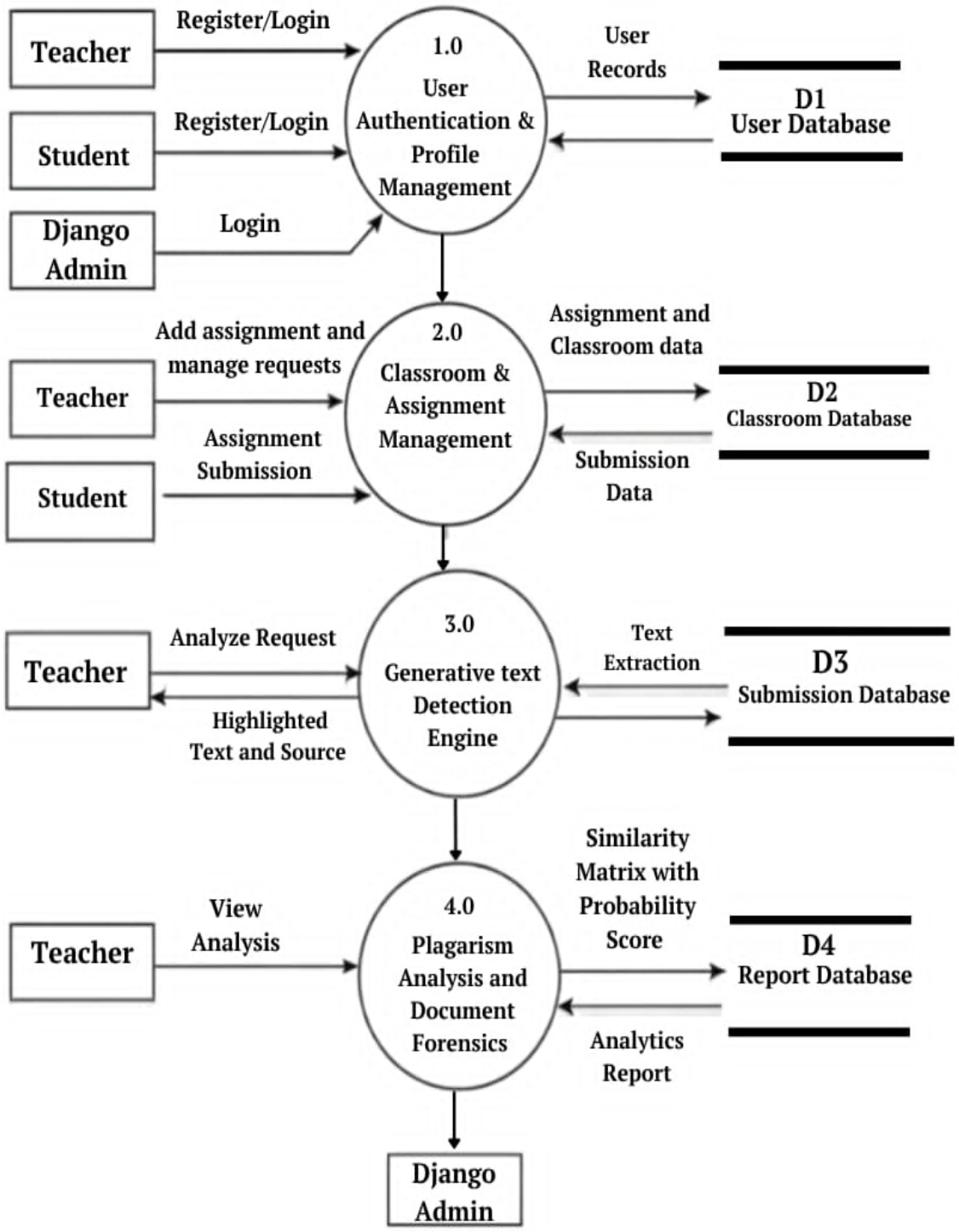


Figure 4.3: Level 1 Data Flow Diagram of the System

User Authentication and Profile Management handles the registration and login activities of both the Teacher and Student. The Teacher and Student provide their registration or login details to this process, while the Django Admin can also log in through it. The process stores and retrieves user records from the User Database (D1) and, after successful authentication, passes the user to the Classroom & Assignment Management process.

Classroom and Assignment Management manages classroom information, assignments, requests, and student submissions. The Teacher uses this process to add assignments and manage classroom requests, while the Student submits assignments through it. The process stores assignment and classroom information in the Classroom Database (D2) and retrieves submission data when required. It also sends relevant classroom and assignment information to the Generative Text Detection Engine for further analysis.

Generative Text Detection Engine analyzes submitted text to detect whether it may have been generated by AI and extracts relevant text information. The Teacher sends an analysis request to this process, and the engine returns highlighted text and its source for review. It also interacts with the classroom Database (D3) by retrieving text extraction data and storing or processing the extracted text. The analysis results are then passed to the Plagiarism Analysis and Document Forensics process for further examination.

Plagiarism Analysis and Document Forensics performs detailed plagiarism and document analysis using the information received from the Generative Text Detection Engine. The Teacher can request and view the analysis results through this process. It generates a similarity matrix with probability scores and stores the results in the Report Database (D4). The process can retrieve analytics reports from the database and provides the final analysis for the Teacher, while the Django Admin can also interact with this process for administrative monitoring and management.

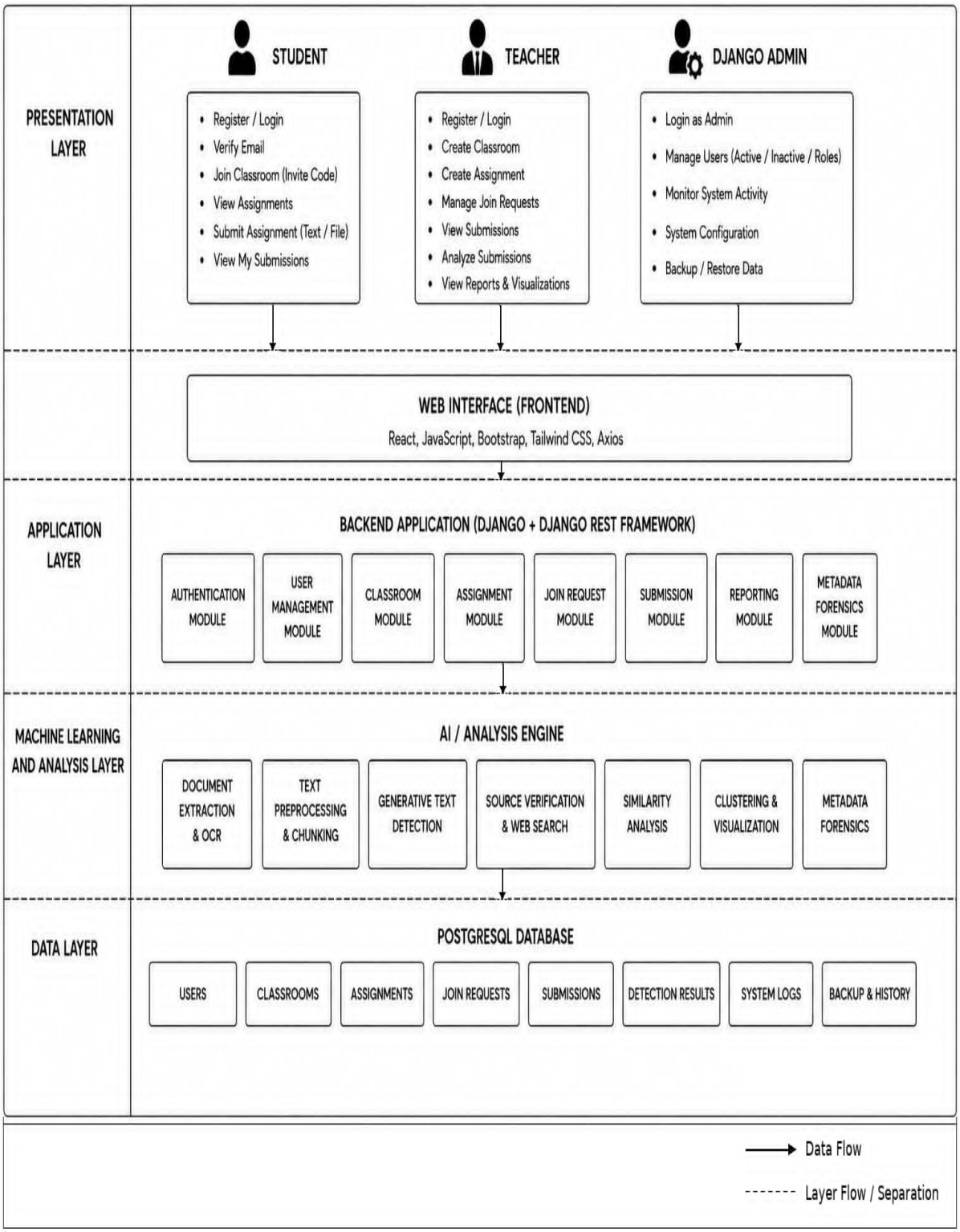


Figure 4.4: Architecture diagram of the system

It is a multi-layer system architecture diagram that shows how the different parts of the system are organized and how data flows from the users through the application and AI analysis components to the database.

The Presentation Layer represents the three main users of the system: Student, Teacher, and Django Admin. It shows the major operations available to each role, such as registration,

classroom and assignment activities, submission analysis, report viewing, user management, system monitoring, and data backup or restoration.

The Web Interface (Frontend) acts as the interaction layer between the users and the system. It is developed using technologies such as React, JavaScript, Bootstrap, Tailwind CSS, and Axios. User actions are passed from the frontend to the backend application through this layer.

The Application Layer contains the main backend modules implemented using Django and Django REST Framework. These modules handle authentication, user management, classrooms, assignments, join requests, submissions, reporting, and metadata forensics. They process requests from the frontend and coordinate the appropriate system functions.

The Machine Learning and Analysis Layer contains the AI and analysis engine, which performs the core document analysis tasks. It includes document extraction and OCR, text preprocessing and chunking, generative text detection, source verification and web search, similarity analysis, clustering and visualization, and metadata forensics. This layer is responsible for analyzing submitted documents and producing meaningful detection and plagiarism-related results.

The Data Layer uses a PostgreSQL database to store and manage the system's persistent information. It contains data related to users, classrooms, assignments, join requests, submissions, detection results, system logs, and backup history. The database supports the application and analysis layers by providing the required data for processing and storing generated results.

4.3.3 Technology stack used

Table 4.1: Technology Stack

Layer	**Technologies**	**Purpose**
Frontend	React 19, Vite, React Router, Axios, Tailwind CSS	UI, routing, styling, API communication.
Backend	Python, Django, Django REST Framework	APIs, models, business rules, serialization.
Authentication	Simple JWT	Access/refresh tokens and protected APIs.
Database	PostgreSQL	Persistent application data.
Document tools	python-docx, pypdf/PyPDF2, python-pptx	Text and metadata extraction.
OCR	Tesseract, EasyOCR, TrOCR, optional Google Vision	Reads scans/images.
Analysis	joblib model, NumPy, custom vector logic	AI-like text and similarity scoring.
Source lookup	Requests, Firecrawl and search fallbacks	Finds possible online sources.
Visualization	Matplotlib, Seaborn, Pandas	Heatmap and cluster plots.

4.3.4 System Flowchart

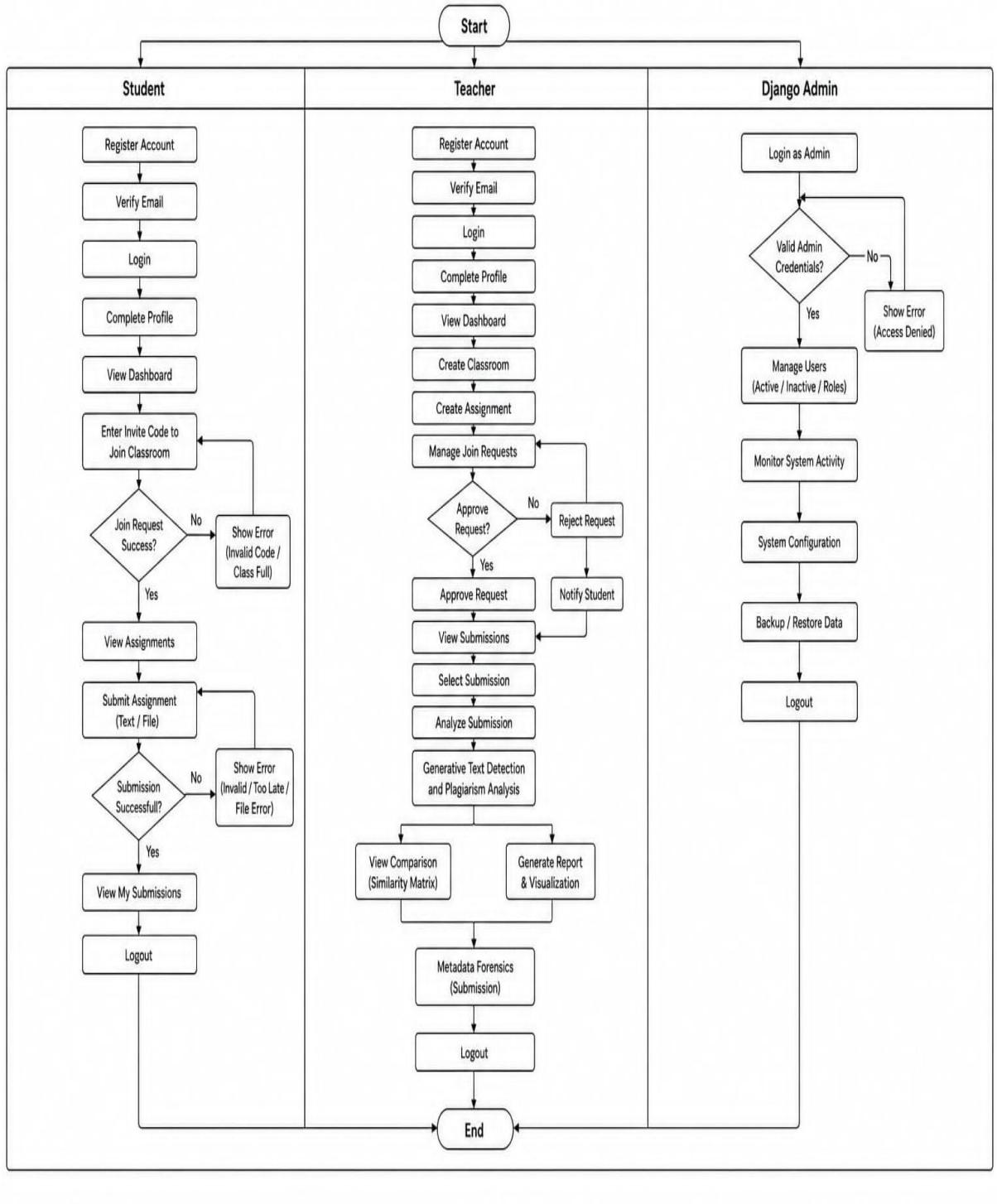


Figure 4.5: System Flowchart Showing Student, Teacher and Admin Workflows

The system flowchart in Figure presents the complete operational workflow of the Generative Text Detection, Plagiarism Analysis, and Document Forensics System through three separate user lanes: Student, Teacher, and Django Admin. The flow begins from a common start point and then follows different activities according to the responsibilities of each role.

The Student workflow starts with account registration, followed by email verification, login,

and profile completion. The student then accesses the dashboard and enters an invite code to join a classroom. The system checks whether the join request is successful; if the code is invalid or the classroom is full, an error is displayed. After successfully joining, the student can view available assignments and submit an assignment as text or a file. The submission is validated, and unsuccessful submissions due to issues such as invalid files or late submission return an error. Successful submissions are stored, after which the student can view their submissions and log out.

The Teacher workflow begins with registration, email verification, login, and profile completion before accessing the dashboard. The teacher can create classrooms and assignments and manage student join requests. The system provides a decision point for each request, allowing the teacher to either approve or reject it. Approved students can proceed within the classroom, while rejected requests result in a notification being sent to the student. The teacher can then view submissions, select a specific submission, and initiate the analysis process. The system performs Generative Text Detection and Plagiarism Analysis, after which the teacher can view the similarity matrix, generate reports and visualizations, and perform metadata forensics on the submitted document.

The Django Admin workflow is focused on administrative and maintenance activities. The administrator first logs in through the admin panel, where the system validates the administrator's credentials. If the credentials are invalid, an access-denied error is displayed and the administrator is returned to the login stage. With valid credentials, the administrator can manage users, monitor system activity, configure system settings, and perform backup or restoration of system data. The administrator then logs out after completing the required administrative operations.

The separate workflows demonstrate how the system handles authentication, classroom management, assignment submission, analysis, administrative control, and error conditions for different user roles. Each lane follows its own sequence of activities and decision points, while the workflows ultimately converge at the End node, representing the completion of the overall system process.

4.3.5 Data Preparation and Machine Learning Workflow

The data collection and integration process consists of two parallel pipelines: the AI detection model dataset preparation pipeline and the external content collection pipeline for plagiarism verification. In the AI detection pipeline, the Kaggle DAIGT v2 dataset containing human-written and AI-generated text samples is collected and cleaned by removing duplicate records, missing values, and inconsistencies. The text is then preprocessed through tokenization, lowercasing, and normalization before being divided into training (80

Simultaneously, the plagiarism verification pipeline gathers technical and educational content from online sources using the Firecrawl Search API. Relevant website content, including ar-

ticles, tutorials, blogs, and code snippets, is extracted and stored for comparison. Similarity matching techniques such as SequenceMatcher and fuzzy matching are then applied to compare the submitted text with the collected content and calculate similarity scores. Finally, the outputs of both the AI text detection model and the plagiarism verification pipeline are integrated into a unified originality analysis framework. This framework provides comprehensive originality assessment by combining AI-generated text detection with plagiarism verification, enabling accurate analysis and effective decision support for users.

5. Results and Discussion

This chapter presents the results of the OriginalityGuard system implementation. The project has reached full integration status, with the frontend, backend, REST API, OCR module, AI-text detector, source-verification service, and metadata-forensics engine implemented and functioning together as a single role-based platform.

5.1 Experiment Setup

5.1.1 Dataset and Preprocessing

The AI-text detector operates on the persisted model ml_models/originality_model.joblib, which is loaded at request time to score submitted text. Input text is normalized and split into paragraph- or sentence-level chunks prior to scoring, and large unstructured OCR output is further segmented into approximately 100–120 word blocks. A separate corpus of PDF, DOCX, PPTX, HTML, and image-based documents was used to exercise the extraction and OCR pipeline across formats.

5.1.2 Feature Extraction

Chunk-level AI-likelihood is produced directly by the persisted joblib model as a normalized probability between 0 and 1 per chunk, without requiring a separate manual feature-engineering step at inference time. For pairwise similarity, three independent feature signals are extracted per document pair: Jaccard token-set overlap, TF-IDF cosine similarity, and a custom semantic weighted token-overlap score.

5.2 Software Development Approach

5.2.1 Frontend Implementation

The React 19 (Vite) frontend delivers a role-based interface with distinct Student and Teacher dashboards. `SubmissionDetail.jsx` renders chunk-level AI-probability bars, triggers source verification, and polls for background results. `MetadataForensics.jsx` renders similarity clusters, the pairwise similarity matrix, metadata fields, heatmaps, and scatter plots. JWT-based authentication with automatic token refresh is functioning across all protected routes.

5.2.2 Backend Architecture

The Django REST Framework backend is organized into three functioning apps –

`authentication`, `classroom`, and `analysis_engine` – with role-filtered querysets

enforcing that students, teachers, and administrators access only permitted data. Database integrity rules are enforced and confirmed working: unique classroom invite codes, one join request per student per classroom, one submission per student per assignment, and rejection of submission create/update after the assignment deadline.

5.2.3 API Development

The REST API is fully implemented and reachable, covering registration, verification, JWT login/refresh, profile management, classroom and assignment CRUD, join requests, submission handling, AI analysis (`/api/analyze/`), source verification (`/api/verify-sources/`), and batch/per-assignment plagiarism-forensics reporting. The analysis endpoint returns a responsive result by performing AI-text scoring only, while source verification runs separately in a background thread and is polled by the frontend every three seconds for up to eight attempts.

5.3 Model Deployment

5.3.1 AI Text Detection Model

The joblib-based detector is deployed and integrated into the submission-analysis pipeline. For each analyzed submission, it produces per-chunk probabilities, an AI-like flag per chunk, total and AI-flagged chunk counts, an average probability, an overall AI-text percentage, and a summary verdict, all persisted as a `DetectionResult` JSON record. The frontend renders these as green/amber/red chunk bars, allowing a teacher to inspect flagged evidence directly rather than relying on a single aggregate output.

5.3.2 Optical Character Recognition Integration

The OCR pipeline – Tesseract, EasyOCR, the TrOCR handwritten-text model, and optional Google Cloud Vision – is deployed and successfully extracts text from scanned PDFs and image-based submissions after preprocessing (grayscale conversion, contrast enhancement, thresholding, upscaling, sharpening). Extracted text is cached in

`Submission.extracted_text` and fed into the same AI-detection and similarity pipeline used for digitally submitted text, giving consistent downstream treatment regardless of submission format.

5.3.3 Plagiarism Verification Pipeline

The similarity engine is deployed and computes, for every submission pair, Jaccard similarity, TF-IDF cosine similarity, and semantic weighted token similarity, combined into an overall similarity score as their mean. Chunk-level matches are also recorded, allowing a strong local match to be surfaced even when whole-document similarity is low. Source verification is deployed as an on-demand service querying Bing, DuckDuckGo, Google, and Firecrawl-related paths, returning candidate URLs and match-confidence scores. Metadata forensics is deployed and reads DOCX/PDF author, title, creator, timestamps, edit/save/revision counts,

and computes a SHA-1 origin-signature hash and a weighted 0–1 originality score used to cluster submissions sharing an origin signature or exceeding a 0.25 similarity threshold.

5.4 Testing and Evaluation

5.4.1 Backend Test Coverage

Automated backend tests in `apps/analysis_engine/tests.py` confirm correct behavior of the following components: similarity report construction, highlighting and chunk comparison, chunk splitting and merging, document-forensics output generation, OCR result selection, file-like text extraction, OCR text normalization, cached-text reuse, and analysis of long OCR-derived text. All tested components pass, validating the analysis pipeline independent of the user interface.

5.4.2 OCR Extraction Results

The OCR pipeline correctly extracts text across the supported formats (PDF, DOCX, PPTX, HTML, and image files) and correctly falls back between embedded-text extraction and image-based OCR engines depending on file type. Preprocessing steps visibly improve extraction on lower-quality scans. Formal word-error-rate or character-accuracy benchmarking against a labeled ground-truth set has not yet been performed.

5.4.3 System Integration Results

End-to-end functional testing confirms the complete workflow operates correctly: account registration through verification, classroom creation and join-request approval, assignment creation, submission upload, AI-report generation, on-demand source verification with background polling, and batch/assignment-level metadata-forensics reporting with cluster, matrix, heatmap, and scatter-plot visualization. The separation of the fast analysis endpoint from the slower, explicitly triggered source-verification step keeps the submission-review flow responsive.

5.4.4 Evidence Presentation Results

The chunk-level probability bars, similarity heatmaps, and cluster views give a teacher visibility into the evidence behind each flag rather than a single opaque score. Origin-signature hash matches and the weighted originality score are surfaced as supporting evidence rather than a final verdict, consistent with the system's role as an originality-assistance tool that flags submissions for instructor review.

6. Conclusion

This project successfully developed OriginalityGuard, a web-based academic originality assistance system that integrates classroom management with document analysis and forensic evidence. The system allows teachers to create classrooms and assignments, manage student access, receive submissions, and review analysis reports, while students can securely register, join classrooms, and submit their work before assignment deadlines. Its React frontend, Django REST backend, PostgreSQL database, and JWT-based authentication provide a structured and role-based environment for academic submission management.

The system combines multiple complementary techniques to support originality review. It extracts text from digital documents and scanned files using document parsers and OCR methods, then performs chunk-level AI-text probability analysis to present interpretable indicators. It also supports on-demand online source verification, pairwise similarity analysis using Jaccard similarity, TF–IDF cosine similarity, and weighted token similarity, as well as PDF and DOCX metadata forensics. The generated reports, similarity matrices, clusters, heatmaps, and metadata records enable teachers to examine suspicious patterns with supporting evidence instead of relying on a single score.

A key contribution of the project is its evidence-oriented design. AI-text probabilities, source matches, document similarity, and metadata anomalies are presented as indicators for instructor review rather than as automatic proof of plagiarism or AI use. This approach recognizes the limitations of OCR quality, probabilistic classification, online search coverage, and editable document metadata, while preserving human judgement as the final basis for academic decisions.

Overall, OriginalityGuard demonstrates that classroom workflows, AI-text indicators, plagiarism-related similarity analysis, OCR, source verification, and document forensics can be integrated into one practical platform. The completed system provides a useful foundation for supporting academic integrity by helping educators prioritize submissions for review, investigate possible concerns systematically, and make informed decisions using transparent analytical evidence.

7. Limitations and Future Enhancements

7.1 Limitations

- AI-text detection produces probability-based results that may vary with the trained model, writing style, document length, and evolving generative AI systems.
- Text extraction and OCR accuracy may decrease for low-quality scans, noisy images, complex layouts, mathematical content, and unclear handwriting.
- Online source verification depends on the availability, accessibility, and indexing of external sources and cannot guarantee detection of all relevant sources.
- Source matches indicate possible evidence but do not conclusively prove plagiarism, while the absence of a match does not guarantee originality.

7.2 Future Enhancements

- AI-text detection can be improved using larger, diverse datasets and comprehensive performance evaluation.
- Multilingual support, including Nepali, can be added along with embedding-based semantic similarity for better paraphrase detection.
- Scalable asynchronous processing can be introduced using Celery and Redis, along with progress tracking, notifications, and downloadable reports.
- The platform can be extended with rubric-based grading, teacher comments, audit logs, submission history, and examiner analytics.
- Security and deployment can be strengthened through secure cloud storage, file validation, malware scanning, HTTPS, and improved access controls.

References

[1] A. Schulz et al. Hierarchical stylometric analysis for authorial intent identification. *Journal of Academic Integrity Research*, 2024.

[2] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[3] B. Guo et al. Detecting synthetic text via perplexity and burstiness metrics in LLMs. In *International Conference on Machine Learning Applications*, 2025.

[4] J. Smith and R. Jones. Robust document digitization: Integrating OCR and HTR for academic forensics. *Journal of Digital Archiving and Document Processing*, 2023.

[5] Y. Wang et al. Digital forensic frameworks for metadata integrity in OOXML and PDF documents. *IEEE Transactions on Information Forensics and Security*, 2025.

Appendices